



@thebinarytechnology



Abhay Tiwari

SAVE THIS
FOR LATER



TOP 50 BACKEND INTERVIEW QUESTIONS

5
PAGES
Q1-Q50

500+
Important
Concepts

Well
Structured
& Easy to
Understand

Perfect for
SDE, Backend
& Full Stack
Roles

Interview
Ready
Be Confident.
Be Prepared.

★ Ace Your Next Backend Interview! ★



WHAT'S INSIDE?

Page 1



Backend
Fundamentals

Q1 - Q10

Page 2



Authentication
& API Security

Q11 - Q20

Page 3



Database
& SQL

Q21 - Q30

Page 4



System Design
& Scalability

Q31 - Q40

Page 5



Architecture &
Best Practices

Q41 - Q50

PREPARE SMART.
CRACK CONFIDENTLY.

- ✓ Essential for Interviews
- ✓ Curated with Industry Standards
- ✓ Save Time • Learn Faster
- ✓ Boost Confidence



LIKE

If you find this useful



COMMENT

Your favorite topic



SAVE

For your interview prep

FOLLOW @thebinarytechnology FOR MORE SUCH CONTENT!



TOP 50 BACKEND INTERVIEW QUESTIONS



Page 1 - Backend Fundamentals (Q1-Q10)

1



Q. What is a REST API?

Ans: REST (Representational State Transfer) API is an architectural style that uses HTTP methods to perform CRUD operations on resources using URLs.

CRUD with REST

GET	→ Read
POST	→ Create
PUT	→ Update
DELETE	→ Delete

2



Q. REST vs SOAP?

Ans: REST is lightweight, uses JSON, HTTP, and is easy to use. SOAP is protocol-based, uses XML, has strict standards and supports advanced security.

REST	SOAP
Protocol: HTTP	Protocol: HTTP / SMTP
Data Format: JSON	Data Format: XML
Speed: Fast	Speed: Slow
Complexity: Low	Complexity: High

3



Q. What are HTTP methods?

Ans: GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD.

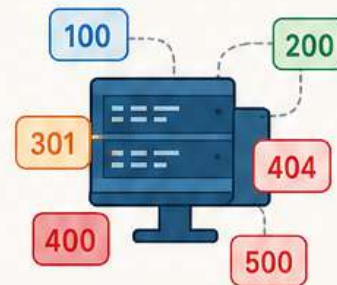
4



Q. Common HTTP status codes?

Ans:

- 1xx** (Informational) → Request received, continuing process
- 2xx** (Success) → 200 OK, 201 Created
- 3xx** (Redirection) → 301 Moved, 304 Not Modified
- 4xx** (Client Error) → 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
- 5xx** (Server Error) → 500 Internal Server Error, 502 Bad Gateway

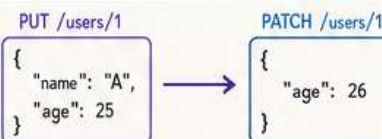


5



Q. PUT vs PATCH?

Ans: PUT → Replaces the entire resource.
PATCH → Updates only the provided fields / partial update.



6



Q. What is idempotency?

Ans: Idempotency is the property of an operation where making the same request multiple times produces the same final state as making it once.

7



Q. Stateless vs Stateful?

Ans: Stateless → Server does not store client state. Each request is independent. (REST APIs)
Stateful → Server stores client state (e.g., Session-based authentication).

8



Q. What is CORS?

Ans: CORS (Cross-Origin Resource Sharing) is a security feature that allows or restricts web pages from making requests to a different domain.



9



Q. What is API versioning?

Ans: API versioning is a way to manage changes in APIs while adding new features without breaking existing clients.
Example: /api/v1/users

Ways to Version:

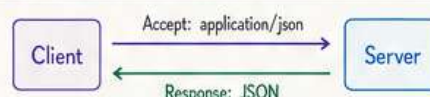
- URL (/v1/users)
- Header (Accept-Version)
- Query Param (?version=1)

10



Q. What is Content Negotiation?

Ans: Content Negotiation is the HTTP mechanism through which a client and server agree on the best representation of a resource (such as JSON, XML, HTML, or images) using request headers like Accept.



01



Fundamentals

02



Auth & /Security

03



Database & SQL

04



System Design

05



Architecture

TOP 50 BACKEND INTERVIEW QUESTIONS

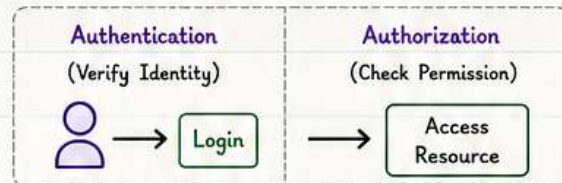


Page 2 - Authentication & API Security (Q11-Q20)



11 Authentication vs Authorization ?

Ans: Authentication verifies **who you are**.
Authorization determines **what you can do**.



12 JWT vs Session ?

Ans: JWT is **stateless** and stored on client (usually in localStorage or cookies).
Session is **stateful** and stored on server.

Feature	JWT	Session
Storage	Client (Token)	Server (Session Store)
State	Stateless	Stateful
Scalability	High	Low (depends on store)
Performance	Fast	Relatively Slow
Security	Secure (if implemented right)	Secure

13 OAuth vs JWT ?

Ans: OAuth is an **authorization framework** that allows access to resources without sharing credentials.
JWT is a token format used to securely transmit information between parties.

OAuth (Protocol / Framework)

- Used for delegated authorization
- Example: Google Login, GitHub Login
- Involves Authorization Server

JWT (Token)

- Used to represent claims
- Compact and self-contained
- Used in OAuth, Custom Auth

14 Access Token vs Refresh Token ?

Ans: Access Token is **short-lived** and used to access resources.
Refresh Token is **long-lived** and used to get a new Access Token.

Feature	Access Token	Refresh Token
Purpose	Access resources	Get new access token
Lifetime	Short (5-15 min)	Long (7-30 days)
If Stolen	Damage is limited	Can issue new access token
Storage	Memory / LocalStorage	HttpOnly Secure Cookie

15 What is HTTPS ?

Ans: HTTPS is HTTP over **TLS/SSL**. It encrypts data between client and server to ensure confidentiality, integrity and authenticity.



16 What is Rate Limiting ?

Ans: Rate Limiting restricts the number of requests a client can make within a specific time window to prevent abuse.



Example:

100 requests / 15 minutes / IP

17 CSRF vs XSS ?

Ans: CSRF (Cross-Site Request Forgery): Attacker tricks user browser to make **unwanted requests**.
XSS (Cross-Site Scripting): Attacker injects **malicious scripts** that run in the user's browser.

CSRF

- Targets authenticated users
- Uses user's session/cookies
- Example: Transfer money

XSS

- Targets other users
- Injects malicious scripts
- Example: Steal Cookies

18 What is SQL Injection ?

Ans: SQL Injection is an attack where malicious SQL queries are inserted via input fields to manipulate the database.



Unsafe Query:

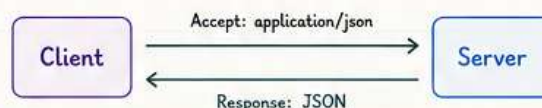
SELECT * FROM users WHERE name = "" + userInput + ""

Safe Approach:

Use Prepared Statements / Parameterized Queries

19 What is Content Negotiation ?

Ans: Content Negotiation is the HTTP mechanism through which a client and server agree on the best representation of a resource (such as JSON, XML, HTML, or images) using request headers like **Accept**.



20 API Keys vs OAuth ?

Ans: API Keys are simple tokens used for identifying the client.

Feature	API Key	OAuth
Purpose	Identify Client	Authorize Access
Security	Less Secure	More Secure

01



Fundamentals

02



Auth & Security

03



Database & SQL

04



System Design

05



Architecture



Save this for your interview prep & share with your friends!





TOP 50 BACKEND INTERVIEW QUESTIONS



Page 3 – Database & SQL (Q21-Q30)



01



Fundamentals

02



Auth & Security

03



Database & SQL

04



System Design

05



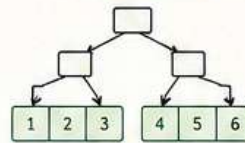
Architecture

21 Q. What is the difference between Clustered and Non-Clustered Index?

Ans: Clustered Index stores the actual table rows in sorted order. Only ONE clustered index per table because the data itself is physically ordered.

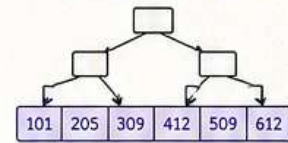
- Non-Clustered Index stores a separate lookup structure containing pointers to actual rows. Multiple non-clustered indexes can exist.

Clustered Index



Data rows stored in sorted order (by index key). Only one per table.

Non-Clustered Index



Index stores pointers (row IDs) to actual data rows. Multiple allowed.

22 Q. What is Indexing?

Ans: Indexing is a technique used to speed up data retrieval by creating a special lookup structure instead of scanning the entire table.

Benefits:

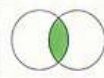
- ✓ Faster SELECT queries
- ✓ Better filtering & searching
- ✓ Reduces disk I/O
- ✓ Slightly slower INSERT/UPDATE (indexes must also be updated)



23 Q. What are the types of SQL JOINS?

Ans: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN and CROSS JOIN.

INNER JOIN



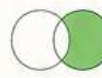
Matching rows only

LEFT JOIN



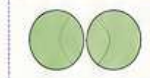
All left + matching right

RIGHT JOIN



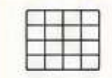
All right + matching left

FULL OUTER JOIN



All rows from both tables

CROSS JOIN



Every possible combination

24 Q. What is the difference between INNER JOIN and LEFT JOIN?

Ans: INNER JOIN returns only matching rows from both tables.
LEFT JOIN returns all rows from the left table and matching rows from the right table. Missing matches become NULL.

INNER JOIN Example

A.id	A.name	B.name
1	John	Sales
2	Alice	HR

Only matching rows from both tables.

LEFT JOIN Example

A.id	A.name	B.name
1	John	Sales
2	Alice	HR
3	Bob	NULL

All rows from left table + matching rows from right.

25 Q. What are the ACID properties?

Ans: Atomicity, Consistency, Isolation, Durability. They ensure reliable processing of database transactions.

Atomicity



All or nothing execution.

Consistency



Database remains in a valid state.

Isolation



Transactions do not interfere with each other.

Durability



Committed data is permanent.

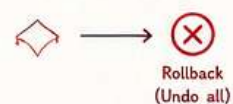
26 Q. What is a Transaction?

Ans: A transaction is a group of SQL operations executed as one logical unit of work. Either every operation succeeds (COMMIT) or everything is rolled back (ROLLBACK).

Example: Money Transfer



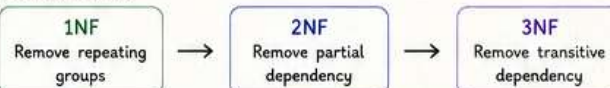
If failure occurs



27 Q. What is Normalization?

Ans: Normalization is the process of organizing database tables to reduce redundancy and improve data integrity.

Normal Forms:



28 Q. What is a Stored Procedure?

Ans: A Stored Procedure is a precompiled collection of SQL statements stored inside the database that can accept parameters and improve code reuse and performance.

```
CREATE PROCEDURE GetUsersByCity(IN cityName VARCHAR(50))
BEGIN
  SELECT * FROM Users WHERE city = cityName;
END;
```



29 Q. What is the difference between View and Table?

Ans: View → Virtual table based on query result. No data stored.
Table → Actual data is stored physically.

Feature	View	Table
Storage	No (Virtual)	Yes (Physical)
Data	From other tables (result)	Stores actual data
Performance	Slower (depends on query)	Faster
Modifiable	Limited	Yes

30 Q. How do you optimize an SQL query?

- ✓ Use indexes on frequently searched columns.
- ✓ Avoid SELECT *.
- ✓ Use proper WHERE clauses.
- ✓ Retrieve only required columns.
- ✓ Avoid unnecessary subqueries.
- ✓ Use LIMIT whenever possible.
- ✓ Use EXPLAIN to analyse execution plan.
- ✓ Optimize JOIN order.





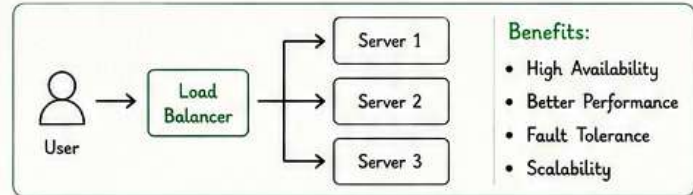
TOP 50 BACKEND INTERVIEW QUESTIONS



Page 4 – System Design & Scalability (Q31–Q40)

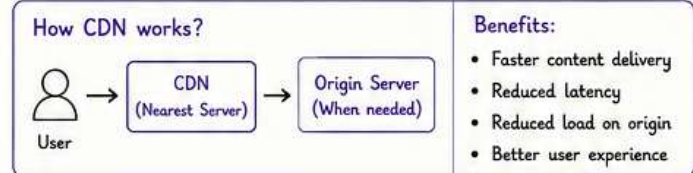
31 Q. What is a Load Balancer?

Ans: Load Balancer distributes incoming traffic across multiple servers to ensure no single server is overloaded and improve availability and reliability.



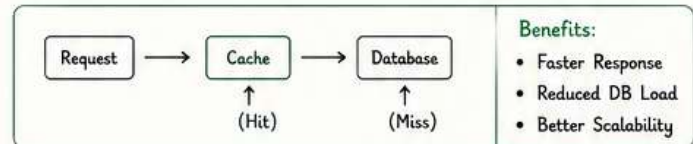
32 Q. What is CDN?

Ans: CDN (Content Delivery Network) is a network of servers distributed globally that deliver content to users from the nearest location.



33 Q. What is Caching?

Ans: Caching stores frequently accessed data in memory so that future requests are served faster.



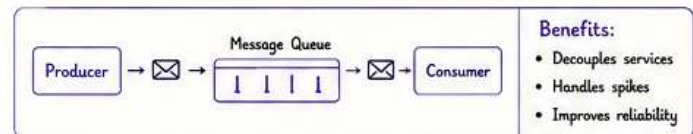
34 Q. What is Redis used for?

Ans: Redis is an in-memory data structure store used for caching, session storage, rate limiting, leaderboards, queues and more.



35 Q. What is a Message Queue?

Ans: Message Queue is a way for applications to communicate asynchronously by sending messages through a queue.



Why use Message Queue?



Asynchronous processing



Better scalability



Reliability & fault tolerance



Load smoothing



Retry & Dead Letter handling

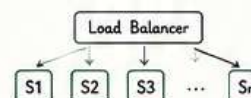
36 Q. Microservices vs Monolith?

Feature	Monolith	Microservices
Architecture	Single deployable unit	Small independent services
Deployment	Deploy all together	Deploy independent services
Scaling	Scale entire application	Scale individual services
Fault Isolation	Low	High
Technology	Same for entire app	Different per service
Complexity	Simple	Complex

37 Q. Horizontal vs Vertical Scaling?

Horizontal Scaling (Scale Out)

- Add more machines/instances
- Distribute load
- Cost-effective
- More fault tolerant



Vertical Scaling (Scale Up)

- Increase power of existing machine
- More CPU / RAM
- Limited by hardware
- Less fault tolerant

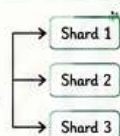


38 Q. What is Database Sharding?

Ans: Sharding is a technique to split large database into smaller parts called shards and distribute across multiple servers.

Sharding Types:

1. Horizontal Sharding → Split rows across servers
2. Vertical Sharding → Split columns across servers



Advantages:

- ✓ Scalability
- ✓ High Availability
- ✓ Faster performance
- ✓ Efficient resource usage

39 Q. What is CAP Theorem?

Ans: In a distributed system, you can have at most two of the following three guarantees:

Consistency (C)

All nodes see the same data at the same time.

Availability (A)

System remains available for every request.

Partition Tolerance (P)

System continues to work despite network partitions or failures.

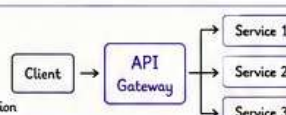


40 Q. What is an API Gateway?

Ans: API Gateway is a single entry point for all client requests. It routes requests to appropriate services and handles cross-cutting concerns.

Responsibilities:

- Routing
- Authentication & Authorization
- Rate Limiting
- Logging & Monitoring
- Caching
- Request/Response Transformation



01

Fundamentals

02

Auth & Security

03

Database & SQL

04

System Design

05

Architecture



TOP 50 BACKEND INTERVIEW QUESTIONS



Page 5 – Backend Architecture & Best Practices (Q41–Q50)



41 Q. What are SOLID Principles?

Ans: SOLID is a set of five design principles that make code more maintainable, scalable and easy to extend.

SOLID Principles

- S** Single Responsibility Principle
- O** Open/Closed Principle
- L** Liskov Substitution Principle
- I** Interface Segregation Principle
- D** Dependency Inversion Principle

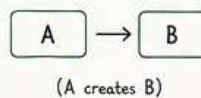
Why Important?

- ✓ Easy to maintain
- ✓ Easy to test
- ✓ Scalable & flexible
- ✓ Fewer bugs

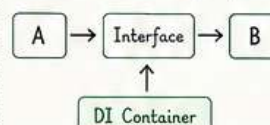
42 Q. What is Dependency Injection?

Ans: Dependency Injection is a design pattern where the dependencies of a class are provided by an external source rather than the class creating them itself.

Without DI (Tight Coupling)



With DI (Loose Coupling)



Benefits:

- ✓ Loose coupling
- ✓ Easy to test
- ✓ More flexibility
- ✓ Easier maintenance

43 Q. What are Design Patterns?

Ans: Design Patterns are reusable solutions to commonly occurring problems in software design. They help write clean, maintainable and scalable code.

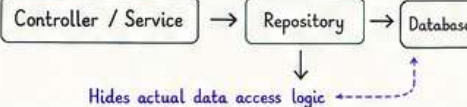
Commonly Used Patterns

- **Singleton** : Ensure a class has only one instance.
- **Factory** : Create objects without exposing instantiation logic.
- **Repository** : Abstraction layer over data access logic.
- **Builder** : Step-by-step construction of complex objects.
- **Observer** : One-to-many dependency between objects.



44 Q. What is Repository Pattern?

Ans: Repository Pattern acts as an abstraction layer between the business logic and data access layer. It helps in cleaner code and easier testing.

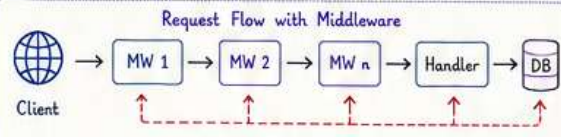


Benefits:

- ✓ Separation of concerns
- ✓ Easier testing (mocking)
- ✓ Swappable data source

45 Q. What is Middleware?

Ans: Middleware is a function that has access to the request and response object in the pipeline. It can execute jstcl, csstgnc code, and easier testing.



Benefits:

- ✓ Modular & reusable
- ✓ Centralized logic
- ✓ Logging, Auth, etc.

46 Q. What is Logging and why is it important?

Ans: Logging is the process of recording events that occur in an application. It helps in debugging, monitoring and auditing.

Log Levels (High —> Low Severity)

- **DEBUG** : Detailed information for debugging
- **INFO** : General operational messages
- **WARN** : Something unexpected but not serious
- **ERROR** : Something failed
- **FATAL** : Critical errors causing shutdown



47 Q. What is Exception Handling?

Ans: Exception Handling is a way to handle runtime errors gracefully without crashing the application.

Try-Catch Example (Most Languages)

```

try {
  // risky code
} catch (Exception e) {
  // handle exception
} finally {
  // cleanup code
}
  
```

Best Practices

- Don't swallow exceptions
- Log the exception
- Return meaningful error messages
- Handle specific exceptions



48 Q. What is Docker?

Ans: Docker is a containerization platform that packages an application and its dependencies into a container so it can run consistently anywhere.

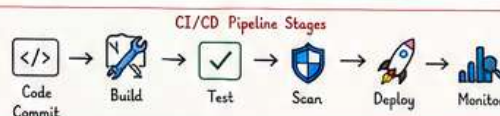


Benefits:

- ✓ Consistency
- ✓ Portability
- ✓ Isolation
- ✓ Efficient resource usage

49 Q. What is CI/CD Pipeline?

Ans: CI/CD stands for Continuous Integration and Continuous Delivery/Deployment. It automates building, testing and deploying applications.



Full Forms:

- **CI** : Continuous Integration
- **CD** : Continuous Delivery
- **CD** : Continuous Deployment

50 Explain a project architecture you built.

Ans: When answering, follow this structure:

How to Answer? (Structure)

- 1 Project Overview - What problem it solves
- 2 Tech Stack - Technologies & frameworks used
- 3 Architecture - High level architecture diagram
- 4 Key Modules - Important components & their role
- 5 Challenges & Solutions - Problems faced & how you solved
- 6 Outcomes - Impact, learnings & results

Sample Architecture (Example)

